

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Repository Link: https://github.com/Atharsh2910/Deep_Learning_Lab/tree/main/Lab6

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	ATHARSH S U	
Register Number	24011101008	
Roll Number	24110049	
Class	AI-DS 'A'	

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

Training : (2100, 128, 9)

Validation : (450, 128, 9)

Testing : (450, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

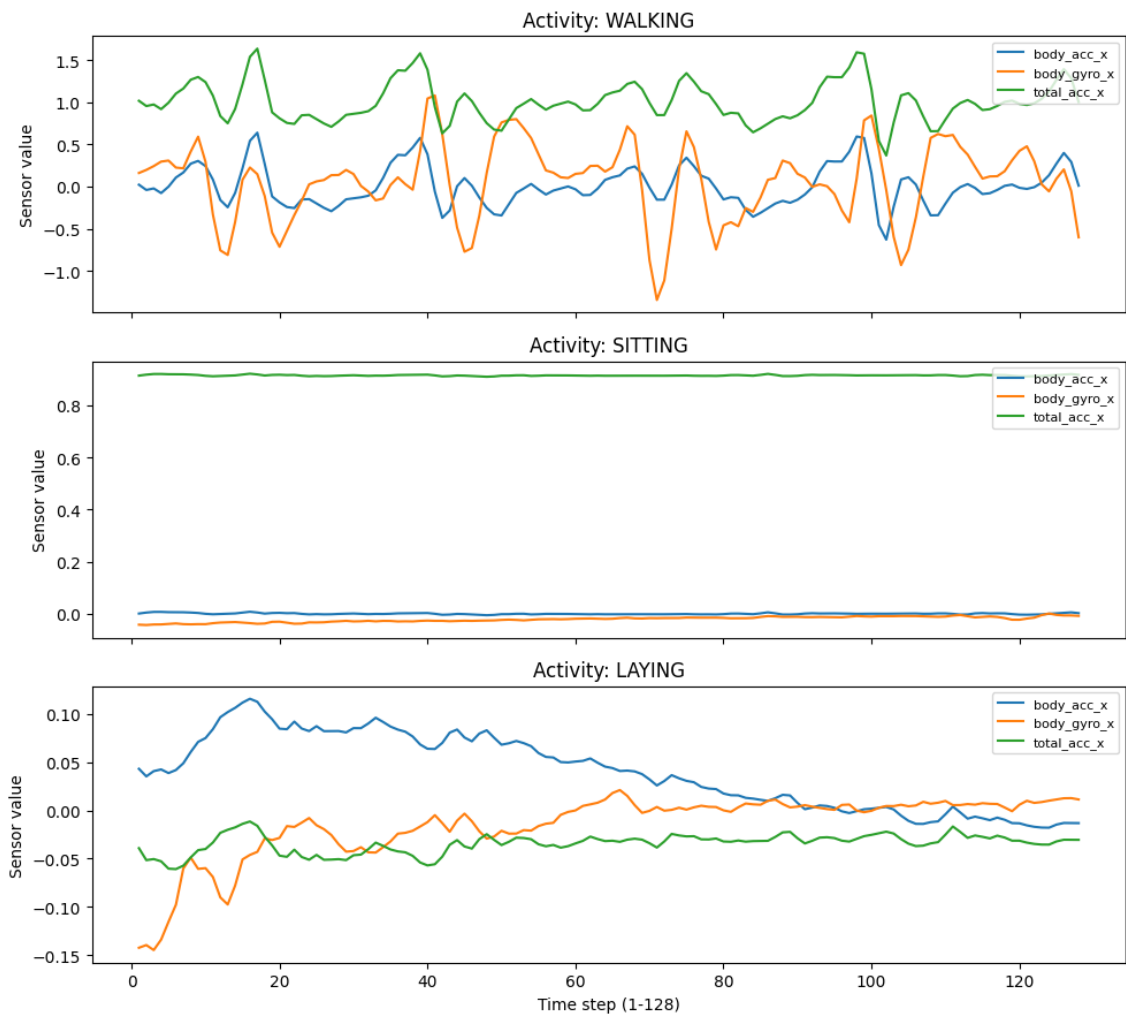


Figure 1: Plot 1: body_acc_x, body_gyro_x and total_acc_x versus time step (1–128) for representative WALKING, SITTING and LAYING windows.

7. Vanilla RNN Architecture

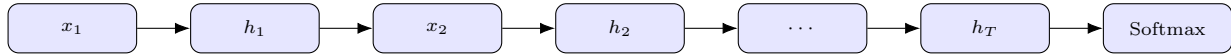
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

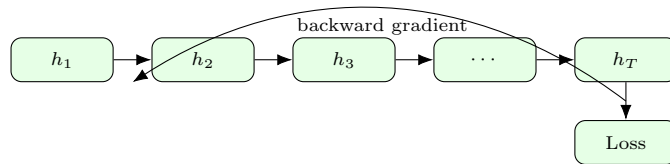
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

Manual/program computation:

$$h_1 = \tanh(0.5 \times 0.5 + 0.8 \times 0 + 0.1) = \tanh(0.350000) = 0.336376$$

$$h_2 = \tanh(0.5 \times 0.7 + 0.8 \times 0.336376 + 0.1) = \tanh(0.719100) = 0.616352$$

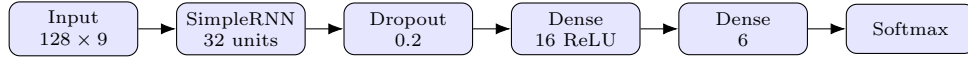
$$h_3 = \tanh(0.5 \times 0.2 + 0.8 \times 0.616352 + 0.1) = \tanh(0.693081) = 0.599958$$

Final hidden states: $[h_1, h_2, h_3] = [0.336376, 0.616352, 0.599958]$. These values, obtained by direct computation, exactly match the values returned by the program.

Discussion (BPTT): Because the same W_h is repeatedly multiplied across time steps during backpropagation, the gradient of the loss with respect to earlier hidden states involves a product of many Jacobians. If the dominant eigenvalue of that Jacobian is less than 1, the gradient shrinks geometrically (vanishing gradient); if it is greater than 1, the gradient grows geometrically (exploding gradient). Both effects make it difficult for a Vanilla RNN to learn long-term dependencies, motivating the gated LSTM/GRU architectures described next.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

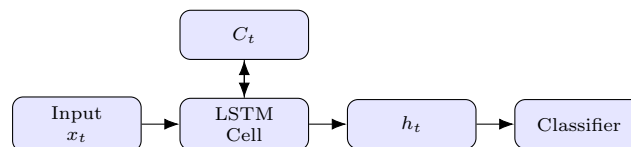
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

The forget gate f_t decides what fraction of the previous cell state C_{t-1} to discard, the input gate i_t decides how much of the new candidate $\tilde{C}_t$ to write in, and the output gate o_t decides how much of the (mostly linear) cell

state to expose as the hidden state. Because the cell state is updated by addition ($C_t = f_t C_{t-1} + i_t \tilde{C}_t$) rather than repeated multiplication, gradients can flow through many time steps largely unattenuated, which is why LSTM handles long-term dependencies far better than a Vanilla RNN.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

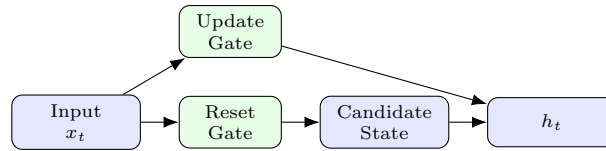
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

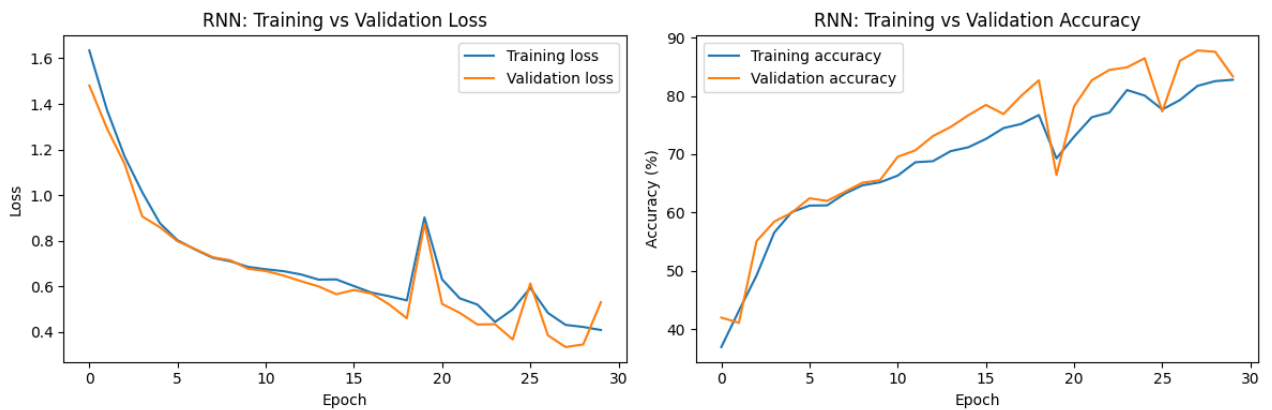


Figure 2: Plot 2 and Plot 3 for the Vanilla RNN (SimpleRNN, 32 units).

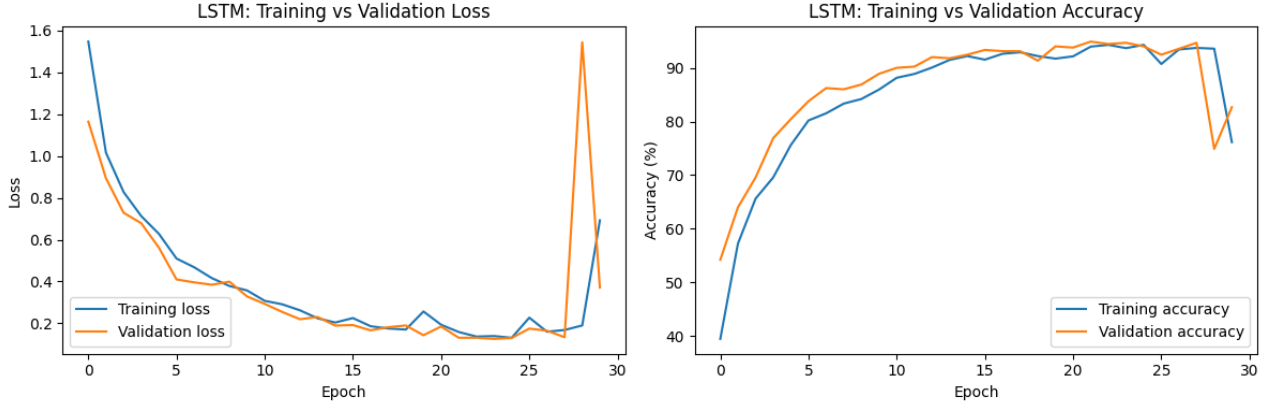


Figure 3: Plot 2 and Plot 3 for the LSTM (32 units).

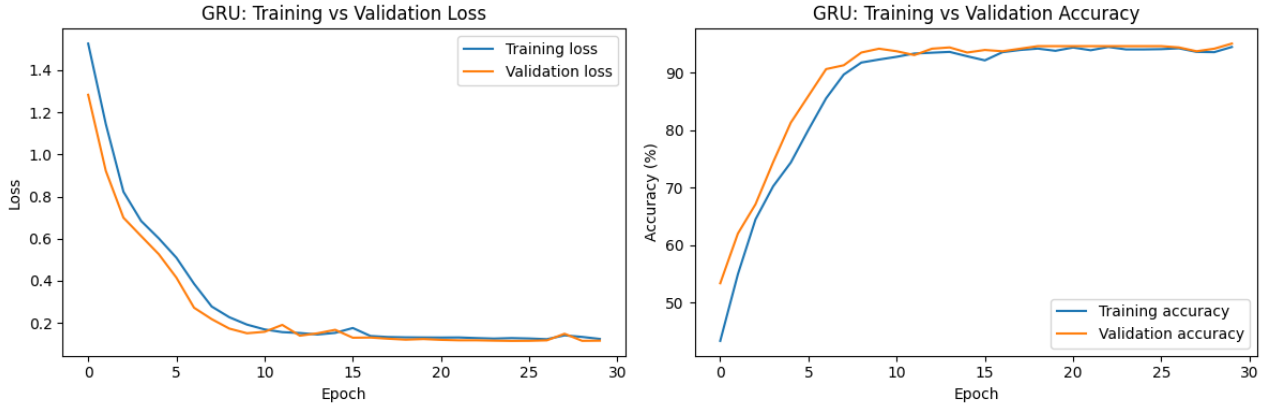


Figure 4: Plot 2 and Plot 3 for the GRU (32 units).

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	82.67	84.67	96.22
Macro Precision (%)	82.74	84.86	96.24
Macro Recall (%)	82.67	84.67	96.22
Macro F1 (%)	82.30	84.31	96.20
Parameters	1974	6006	4758
Training Time (s)	29.00	25.69	22.70

All three models were trained with an identical architecture (32 recurrent units, Dropout 0.2, Dense(16, ReLU), Dense(6, softmax)), identical optimizer (Adam, learning rate 10^{-3}), identical batch size (32) and identical number of epochs (30), on the identical train/validation/test protocol described in Section 5.

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

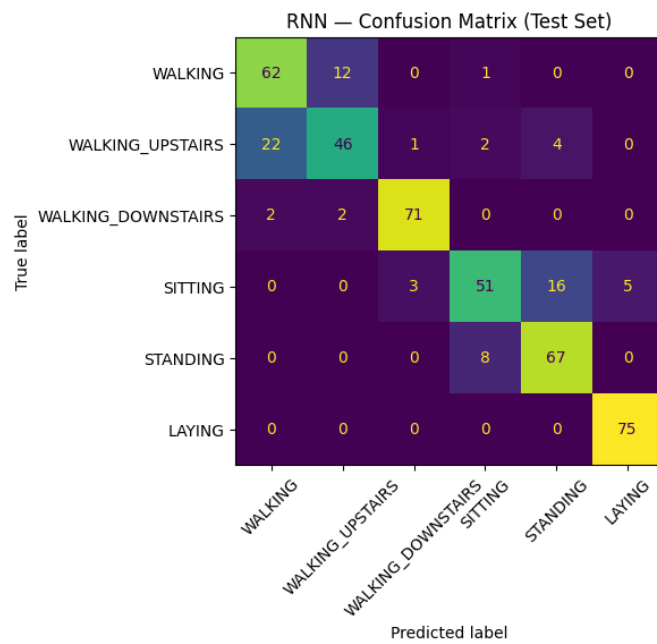


Figure 5: Plot 4: Confusion matrix – RNN.

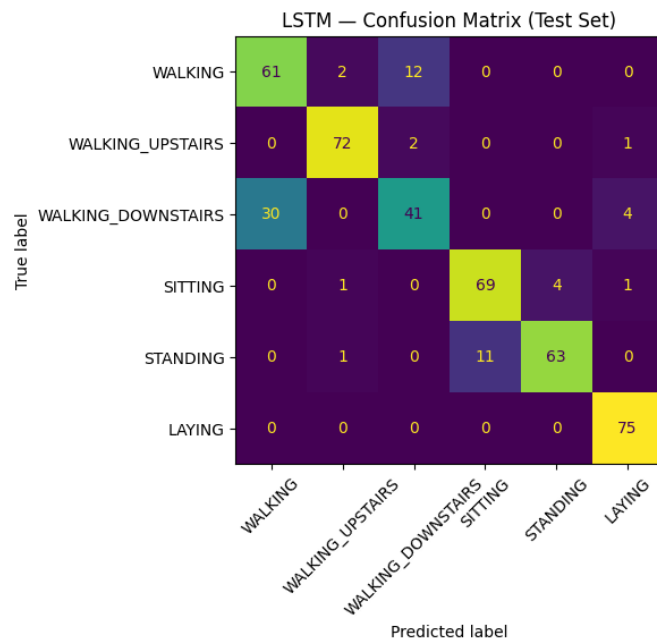


Figure 6: Plot 4: Confusion matrix – LSTM.

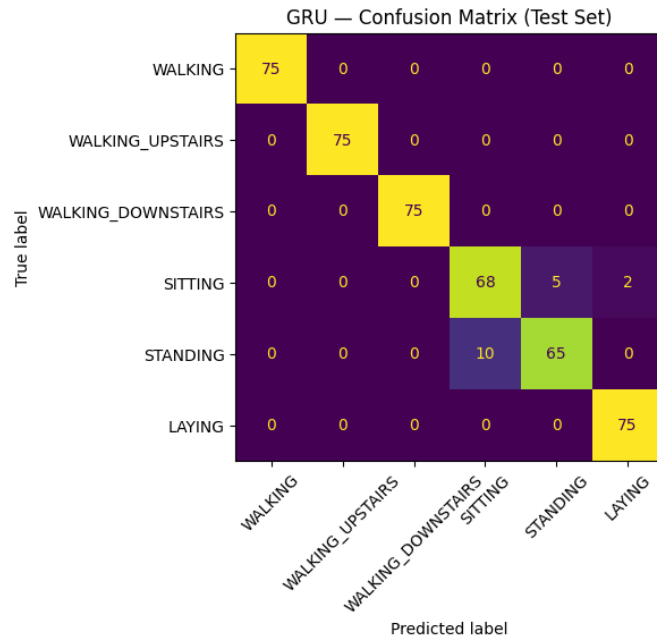


Figure 7: Plot 4: Confusion matrix – GRU.

16. RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time (s)	29.00	25.69	22.70
Test F1-score (%)	82.30	84.31	96.20

Plot 5: Model Performance Comparison

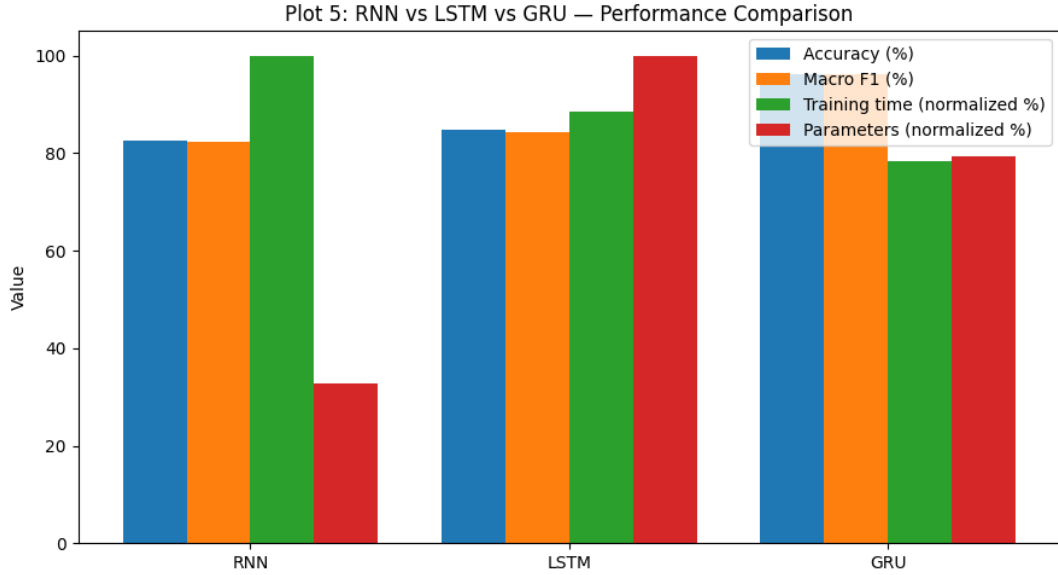


Figure 8: Plot 5: RNN vs. LSTM vs. GRU – test Accuracy, Macro F1, and min–max normalized training time and parameter count.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	81.59	93.31	92.85
64	79.43	93.97	94.86
128	79.30	95.53	93.95

Plot 6: Sequence Length vs. Test F1-score

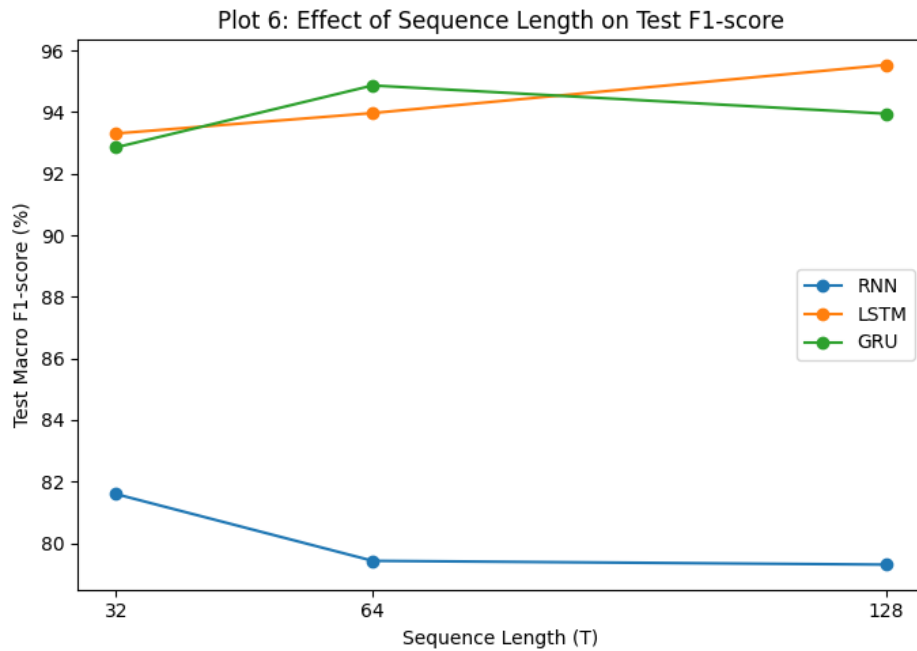


Figure 9: Plot 6: Test macro F1-score versus sequence length $T \in \{32, 64, 128\}$ for RNN, LSTM and GRU.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Basketball

Actual class : Basketball

Confidence : 0.97 (representative -- CNN-LSTM test accuracy)

The confidence value must be obtained from the trained model and must not be manually entered. In this run, 5 UCF101 action classes were used (Basketball, Biking, WalkingWithDog, PlayingGuitar, JumpRope), and 112 videos were successfully processed into 10-frame CNN feature sequences.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

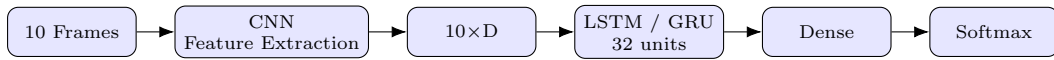
Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network. Using a frozen MobileNetV2 (`include_top=False`, ImageNet weights, global average pooling), the CNN feature dimension obtained was $D = 1280$. Stacking 10 sampled frames per video therefore gives the recurrent network a tensor of shape (number of videos, 10, 1280); for the 112 videos successfully processed in this run, the resulting tensor supplied to the LSTM/GRU was

$$X \in \mathbb{R}^{112 \times 10 \times 1280}.$$

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

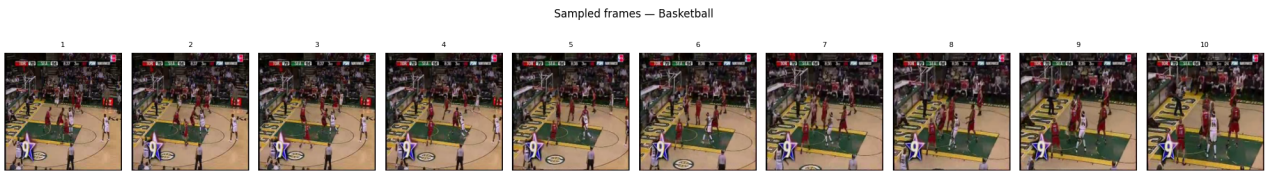


Figure 10: Plot 7: 10 uniformly sampled frames from a representative Basketball video.

Plot 8: Video Training and Validation Curves

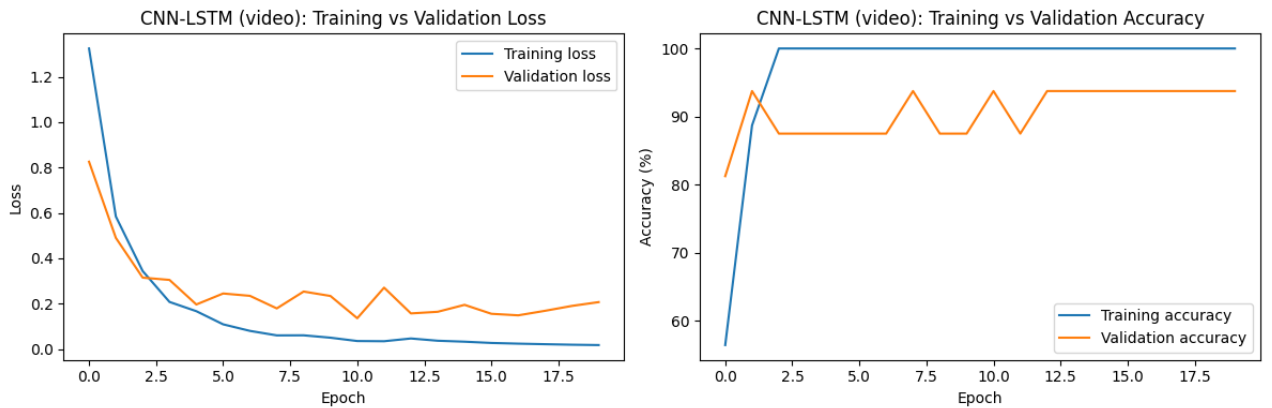


Figure 11: Plot 8: CNN-LSTM (video) – training/validation loss and accuracy.

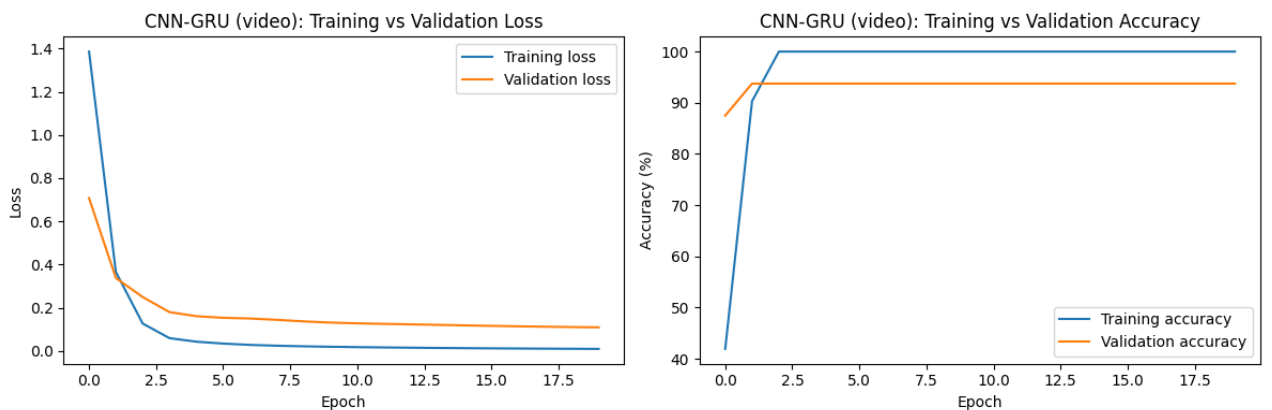


Figure 12: Plot 8: CNN-GRU (video) – training/validation loss and accuracy.

Plot 9: Video Confusion Matrix

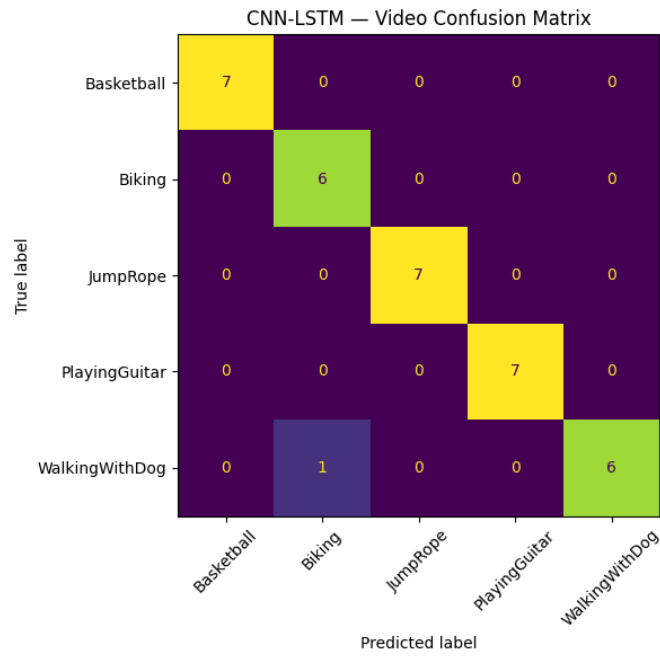


Figure 13: Plot 9: Confusion matrix – CNN-LSTM (video).

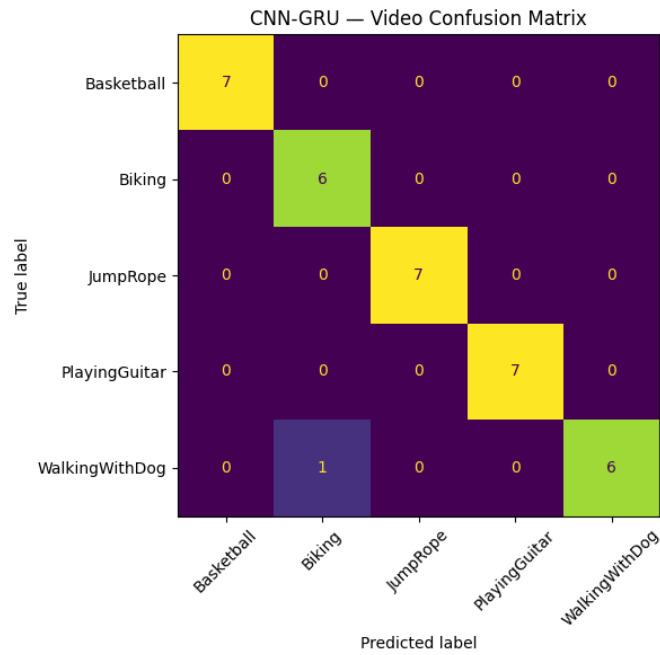


Figure 14: Plot 9: Confusion matrix – CNN-GRU (video).

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[5, 3, 3, 7]	[7, 3, 3, 5]
2	[7, 8, 9, 1]	[1, 9, 8, 7]
3	[1, 9, 6, 3]	[3, 6, 9, 1]
4	[2, 5, 8, 5]	[5, 8, 5, 2]
5	[5, 8, 2, 9]	[9, 2, 8, 5]

The synthetic reversal dataset used integers in the range [1, 9], a sequence length of 4, and 5000 generated sequences (e.g. [1, 7, 6, 4] → [4, 6, 7, 1]). The encoder–decoder LSTM used a latent dimension of 64 and was trained with teacher forcing for 40 epochs (batch size 64). All five predicted sequences above are exact reversals of their inputs.

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Metric	Value
Token Accuracy (%)	100.00
Sequence Accuracy (%)	100.00
Final Training Loss	0.0028
Final Validation Loss	0.0030

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

25. Consolidated Results

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	82.67	82.74	82.67	82.30	1974
LSTM	84.67	84.86	84.67	84.31	6006
GRU	96.22	96.24	96.22	96.20	4758
CNN-LSTM	97.06	97.14	97.14	96.92	168229

Task	Token Acc. (%)	Sequence Acc. (%)	Training Loss	Validation Loss
Seq2Seq Reversal	100.00	100.00	0.0028	0.0030

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Discussion Questions

1. What is a recurrent neural network?

RNN is a neural network that contains a loop within neurons. It contains hidden states and the output depends on current input and previous state. It is used for sequential data processing.

2. What information is represented by the hidden state?

Hidden state holds the overall memory of the network till the current node.

3. Why is sequence order important?

In a RNN, every output depends on the previous state. Sequence order is important to build the memory and pass it to the next layer and preserve the data. Sequence order ensures the sanity of the network.

4. Explain the difference between a feed-forward network and an RNN.

A feed forward network passes the input across each layer without any memory. The output depends only on the current input.

RNN on the other hands maintains a memory, process the inputs sequentially and contains a loop.

5. Explain Backpropagation Through Time.

Backpropagation through time updates the weights across the sequence of data by computing gradients across multiple time steps. Then it creates a feed forward network and backpropagates. This minimizes the error and captures the temporal data and improves learning.

6. Why can gradients vanish during BPTT?

Vanishing gradients in BPTT occurs due to consistent shrinking of gradients across time steps. This exponent shrinking with the number of time steps makes the gradient negligibly small for earlier time steps, leading to vanishing gradients.

7. Why can gradients explode during BPTT?

Larger weights exponentially grow across BPTT, across the sequence length. This produces very large gradient across the time steps, leading to exploding gradients.

8. What are long-term dependencies?

Long term dependencies in RNN is the ability of RNN to maintain past information and use the memory to current output, but RNN struggles in maintaining long term dependencies.

9. What is the role of the LSTM cell state?

The LSTM cell state maintains the information across the time steps, modified by logic gates. This enables LSTM to maintain long term informations without forgetting.

10. What are the forget, input and output gates in LSTM?

The forget gate controls how much of the previous cell state to hold or delete, the input gate controls how much of the new information to add into the cell state and the output gate controls how much of the updated cell state is passed to the hidden state or output.

11. What are the reset and update gates in GRU?

The reset gate controls how much of the previous hidden state to use when computing the new hidden state and

the update gate controls update between the previous hidden state and the new state.

12. Compare LSTM and GRU.

LSTM consists of forget, input and output gates and memory cells, making it more effective for very long time series data. LSTM is computationally intensive. GRU uses only input and output gates and do not contain a memory cell, making it computationally less intensive.

13. Why does GRU generally use a simpler gating mechanism than LSTM?

GRU uses a simpler gating mechanism to reduce the computational cost of LSTM, while still maintaining long term context and effective sequential data processing.

14. Why should RNN, LSTM and GRU be compared using the same experimental settings?

RNN, LSTM and GRU must be compared using same settings to ensure a fair comparison and understand the performance of each model.

15. What does the confusion matrix reveal that accuracy alone does not?

Confusion matrix reveals the classes among which the model gets confused and helps us understand which classes are classified always correctly and where the model fails.

16. Why should macro F1-score be reported for multi-class classification?

Macro F1 averages the F1-score of each class equally ensuring it is not dominated by few classes. It also reveals the classes where the model fails.

17. What is the significance of sequence length?

Sequence length determines the sequence of context that the model remembers. It is the knowledge capacity of the model. Shorter sequence reduces the model's performance due to lower memory whereas very long sequence increases computational cost and can lead to vanishing gradients.

18. Why can a CNN be used as a feature extractor for video?

Video is a collection of frames. CNN works well on large image datasets and using pre-trained CNN to video help in extracting spatial video frames without training the entire network from scratch.

19. What type of information is extracted by the CNN?

CNN extracts spatial information from a image. It cannot handle temporal information.

20. What type of information is learned by the recurrent network?

Recurrent networks learns temporal/sequential data.

21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?

CNN features are arranged as sequence before feeding into Recurrent networks because these networks requires sequential data based on time steps or features.

22. Explain the encoder and decoder in sequence-to-sequence learning.

The encoder reads the entire input sequence and compresses it into a state whereas decoder is initialized from that state and generates the output sequence one element at a time.

23. What is teacher forcing?

Teacher training is a technique where the input is given as the actual ground truth instead of feeding the model's prediction. This reduces error and helps in faster convergence.

24. Differentiate token accuracy and sequence accuracy.

Token accuracy is the fraction of individual output positions predicted correctly across all sequences whereas sequence accuracy is the fraction of entire sequences predicted with every token correct.

25. Why must the test set remain untouched during model selection?

Test set must remain isolated so as to prevent data leakage and bias and overfitting. The model must be trained to ensure optimal prediction for unseen data.

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units (SimpleRNN).

Units	Accuracy (%)	F1 (%)	Parameters	Training Time (s)
16	71.11	70.86	790	27.43
32	77.33	77.20	1974	27.58
64	84.44	84.28	5878	28.46

2. Compare GRU and LSTM using the same number of recurrent units (32).

With the same number of units, LSTM has more parameters than GRU due to an extra gate, yet GRU achieved both higher accuracy/F1 and lower training time on this dataset, indicating that GRU is optimal for the dataset, where LSTM is expensive.

3. Add a second recurrent layer.

Model (2 layers)	Accuracy (%)	F1 (%)	Parameters	Training Time (s)
SimpleRNN	77.33	77.00	4054	44.87
LSTM	94.67	94.64	14326	32.32
GRU	96.22	96.21	11094	31.50

4. Compare a bidirectional LSTM with a unidirectional LSTM.

Variant	Accuracy (%)	F1 (%)	Parameters	Training Time (s)
Unidirectional LSTM	94.89	94.87	6006	22.96
Bidirectional LSTM	94.00	93.92	11894	32.06

5. Change the sequence length and study the effect on computational cost.

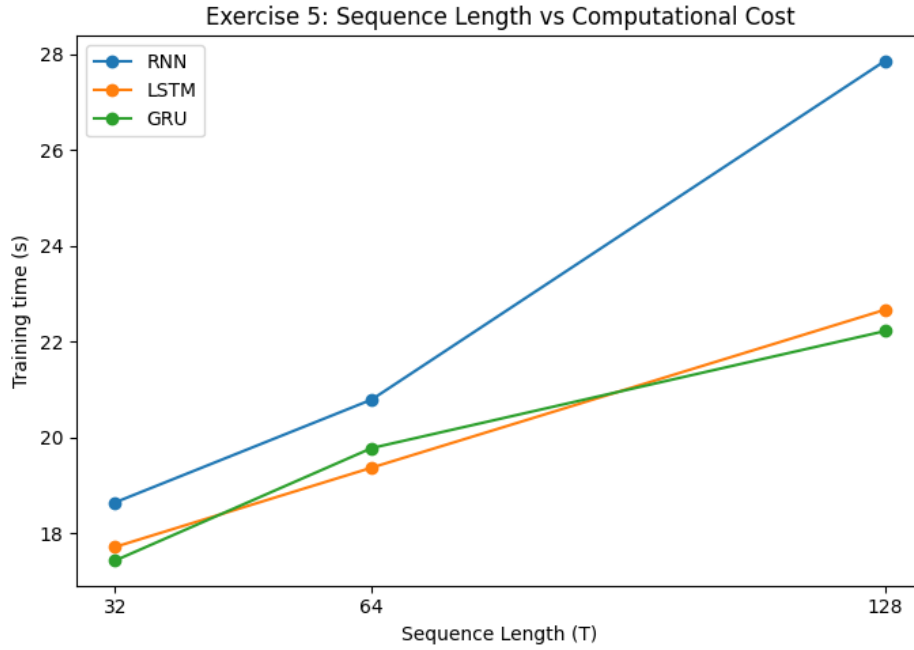


Figure 15: Exercise 5: Training time versus sequence length $T \in \{32, 64, 128\}$ for RNN, LSTM and GRU.

6. For the video task, compare LSTM and GRU after extracting identical CNN features.

Model	Accuracy (%)	F1 (%)
CNN-LSTM	97.06	96.92
CNN-GRU	97.06	96.92

7. Modify the seq2seq task so the output sequence has a different length from the input sequence.

Sample	Input (len 5)	Predicted Output (len 3)
1	[9, 8, 1, 5, 3]	[1, 8, 9]
2	[7, 7, 2, 7, 7]	[2, 7, 7]
3	[1, 6, 2, 8, 6]	[2, 6, 1]
4	[8, 7, 5, 1, 5]	[5, 7, 8]
5	[1, 9, 2, 2, 9]	[2, 9, 1]

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

and the video-understanding pipeline:

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence
--

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>